Weekly Report: Independent Reproduction of ADGR

Full pipeline, full numbers, every figure with a reading guide

Contents

1 Terminology conventions **2**

2 Weekly goals **2**

3 Work done, stage by stage (with the hard part each) **2**

4 Similarities and deviations vs the paper (complete list) **3**

5 Main contributions of this reproduction **4**

6 What each component does (along the pipeline) **4**

7 Full results: ours vs paper **5**

7.1 Table 1 – Graph scale and quality, cell by cell 5

7.2 Table 3 – Retrieval coverage, with the odd-looking arm order explained . 7

7.3 Table 4 – Answer quality, the biggest gap, fully dissected 7

7.4 Table 5 – Five-run judging, both prompts 8

7.5 Table 6 – Cost: cheaper than paper on every axis 9

7.6 Table 7 – Every disabled module hurts, in paper order 10

7.7 Table 8 – The Aescin case: compact yet accurate 10

8 Verdict table: paper vs ours per metric (conclusion each) **11**

9 Strengths, each with its number **12**

10 Weaknesses and limits, each with its number **12**

11 Proposed fixes (evidence + real cost each) **12**

12 Significance and conclusion **12**

1 Terminology conventions

This report keeps the English technical terms, each with a short gloss in parentheses on first use. Every number comes from the real result file (`FINAL_v2.json`) — no estimates, no beautified rounding.

- **chunk** (a text passage; here one article section is one chunk).
- **entity** (a drug, disease, symptom name...), **relation** (an entity pair plus its described link).
- **knowledge graph (KG)** (the entity–relation network).
- **Hit@K** (share of questions answered correctly in the top K), **MRR** and **nDCG@10** (rank-weighted measures: higher ranks score more).
- **RAGAS** (the 4-metric RAG quality suite), **faithfulness** (how closely the answer follows the context), **answer relevancy** (fit to the question), **context recall** (does the context cover everything needed), **context precision** (is the context lean and noise-free).
- **ablation** (disabling exactly one module to measure its importance).
- **anchor** (an entity foothold for expanding search), **edge filter** (the noisy-edge filter), **rerank/reranker** (re-scoring and re-ordering candidates).
- **DSER / SCR / LCR** (share of in-domain entities / valid relations / entities inside the giant connected component).
- **standalone** (a model answering bare, without RAG), **judge** (the scoring model).

2 Weekly goals

- Rebuild the full ADGR system from the paper: corpus, KG, retrieval, evaluation.
- Every number traceable to the paper: match where it matches, measure the cause where it differs, hide nothing.
- Output: the merged numbers + 11 figures + this report, ready to present. Each figure carries its own reading guide: what it shows, how to read it, what to conclude, and how to answer follow-up questions.

3 Work done, stage by stage (with the hard part each)

- **Corpus:** crawled 1,934 medical articles (5 dead URLs, 4 rescued via alternate sources, 1 dropped with a written record). Hard part: 429 rate-limits forced retries spread over many hours.
- **Chunking:** the authors' H2 code (`thesis_src`), so the algorithm matches; 15,837 chunks (min 126, median 800, mean 1,129 chars, ∼357 tokens). 8% missing from 5 dead URLs + doc filtering. Hard part: the chunker is not released with the paper, so this pipeline step depends on external code (documented in the README).

- **KG:** Qwen2.5-14B temp 0 over every chunk → 52,578 entities / 106,279 relations (paper: 49,274 / 106,666); 3.3 distinct entities per chunk on average (densest chunk: 171 extraction hits). Hard part: 15,837 chunks through a GPU for tens of hours with occasional malformed outputs; per-chunk checkpoints plus defensive parsing meant a crash cost exactly the chunk in flight.
- **Retrieval:** 300 questions × 5 arms (full + 4 ablations, each disabling exactly one module per Sec. 5.8) + 2 LightRAG baselines with rerank off per Sec. 5.2. Hard part: one GPU shared four ways (vLLM + ollama + reranker + someone else's 4.4GB training job), hence a per-stage VRAM budget, maxlen cuts, CPU keyword extraction, and keyword memoization (1 call per question instead of 5).
- **Scoring:** Hit/MRR/nDCG (dual gold proxies); the 4 RAGAS components implemented from the original formulas; 4 dims × 5 independent judge runs; per-row tokens and latency. Hard part: one dead API key silently poisoned 123 rows before detection — since then every judge function carries a quota guard: a dead key halts loudly (`os._exit(42)`), never marking a failed row "done".
- **Quality control:** per-question checkpoints (a crash loses ≤1 row); a validation gate after each stage (bad numbers exit 99 loudly, 8 boundaries); every fix verified on the machine with evidence (43 bugs total).
- **Resource budget (measured):** three verified vLLM profiles (32k/0.72; 12k/0.62; 8k/0.50 with maxlen 8192; KV 1.91GB < 2.25GB forced the 12k maxlen down to 8192); one GPU split four ways, so keywords come from CPU; Solar caps at 100 req/min, so judging escalates minimal x3 → low x1 (116 vs 3,273 tokens, 28x cheaper); 429s are retried and failed runs never count toward n (`tab5_5run` pattern).

4 Similarities and deviations vs the paper (complete list)

Matches and deviations are all disclosed; none is hidden in the numbers.

- **MATCHES paper Sec. 5.1 (not substitutions):** Qwen2.5-14B-Instruct -AWQ for generation and KG extraction, `embeddinggemma:300m` embedding (768-dim), Qwen3-Reranker-0.6B reranker, vLLM, temp 0 — same models, same settings. Only the vector store differs (ours: numpy matrices + BM25; paper: NanoVectorDB) while the embedding vectors are identical, so lookup is equivalent.
- **Judge:** solar-pro4 instead of GPT-5.5 (no GPT-5.5 API access; its strictness was measured via the original-prompt cross-check in Table 5).
- **Gold labels:** the authors' manual gold → two proxies: slug-gold (optimistic: correct = a chunk from the same article) and single-gold (pessimistic: correct = the single most similar chunk). Without hand-made labels, paper numbers are never impersonated; both scales reported.
- **Standalone:** the paper's 7 paid models → 7 free models, zero overlap (reference points only, never ranked against the paper).

- **Chunker USES the authors' H2 code** (`thesis_src`, outside the repo, not redistributed): 91.9% match by count; the missing 8% comes from 5 dead URLs + doc filtering (only docs with >200 chars of markdown kept) — input data differs, the algorithm does not. Measured consequences: ∼9.4 extraction hits/chunk (Figure 3) and lower LCR.
- **RAGAS:** reimplemented from the published formulas instead of the official library (to share the solar judge and control cost; standard RAGAS on 300 samples, of which ctx.rec. has data for 275 and ctx.prec. for 46; formulas cross-checked).
- **One-step faithfulness** instead of the standard two-step (far cheaper; impact measured on identical data: 0.41 vs 0.77 — both reported, see Figure 6).
- **Own judging prompt** plus a cross-check with the original prompt (Online Resource 1) on 3,000 judgments (original prompt scores ∼0.08 lower but keeps all rankings — see Figure 8).

5 Main contributions of this reproduction

(1) KG at 92–107% of paper scale *with* all four quality metrics recomputed from raw extraction (DSER 0.913, SCR 0.884, LCR 0.816). (2) Caught and fixed 2 ablation arms computed against Sec. 5.8 (the `no_rerank` arm secretly reranked; the `dense_only` arm secretly mixed graph sources). (3) Reimplemented the 4 RAGAS formulas to the original standard. (4) Cross-checked 2 prompts on 3,000 judgments (∼0.08 scale shift, rankings kept). (5) Decomposed the faithfulness gap by experiment (one-step 0.41, two-step 0.77, paper 0.89). (6) Distilled operating rules from 43 real failures (`AGENTS.md`).

6 What each component does (along the pipeline)

Running example: "Who is Aescin contraindicated for?" (QI288).

- **Crawler:** 1,934 URLs → title, section tree, clean text, source URL. 404/timeout/429 met with backoff retries + alternate sources; only 1 URL abandoned after every documented attempt (recorded, never faked).
- **Cleaner:** strips ads/menus/recommendations/duplicates; keeps titles, sections, URLs for traceability. Encoding audit: 0 mojibake across the corpus.
- **Chunker:** section-aligned splitting, long sections split further while keeping parent metadata (91.9% author-chunker match).
- **Extractor:** each chunk → entities (name, 10+1 type, description) + relations (source, target, keywords, description) + source chunk id, temp 0, delimiter format.
- **Index:** normalized matrices (768-dim for chunks/entities/ relations), BM25 over entity names and descriptions, two-way graph-to-chunk maps (960 MB, 8 files).
- **Keywords:** 3–6 short K_l terms + 1 K_h sentence, lexical fallback on LLM failure, memoized (1 call/question, 5x faster). Aescin example → "Aescin, contraindication, pregnant women, breastfeeding".

- **Anchors:** BM25 top-10 union dense top-10 outside BM25, threshold filter, overlap bonus (entities found by both channels), top-15. Anchored: "Aescin", "hypersensitivity", "pregnant women".
- **Edges:** one-hop neighborhoods around anchors, edge filter, joint anchor-plus-query score, quota m, cap N_{max} (at most 50 edges).
- **Global/direct:** top-n_g relations by broad keywords + top-n_s chunks by the raw question (safety nets outside anchor neighborhoods).
- **Merge:** keep $\lfloor rn/2 \rfloor$ per source, round-robin interleave, dedup by id ($\sim$100-chunk pools).
- **Reranker:** Qwen3-0.6B, automatic GPU/CPU pick, per-row fallback on mid-run OOM, always exactly as many scores as documents (contract verified).
- **Generator:** vLLM Qwen2.5-14B-AWQ temp 0, 400 tokens max.
- **Judges:** Hit/MRR/nDCG (dual proxies), 4 RAGAS components, 4 dims $\times$ 5 runs (minimal x3 $\rightarrow$ low x1 escalation: 116 vs 3,273 tokens, 28x cheaper; quota guard halts loudly), token/latency timers.
- **Merger:** 8 tables + 11 figures; any missing file lands in a pending list, never silently skipped (FINAL pending is empty).

7 Full results: ours vs paper

Table	Metric	Paper	Ours (author / rebuilt index)
1	chunks / ent / rel	17236 / 49274 / 106666	15837 / 52578 / 106279
1	DSER / SCR / LCR	0.936 / 0.928 / 0.906	0.913 / 0.884 / 0.816
3	Hit@1 / Hit@10 full	0.5967 / 0.7100	0.4333 / 0.7933 ; 0.6833*
3	MRR / nDCG@10 full	0.6437 / 0.6606	0.5538* / 0.6118* (0.3849* / 0.4573*)
4	RAGAS / faith / ans.rel / rec / prec	0.8792 / 0.8878 / 0.7124 / 0.9433 / 0.9733	0.64 / 0.41 / 0.90 / 0.64 / 0.86
4	two-step faith	—	0.77 (see analysis)
5	ADGR / Mix / Hybrid	0.7921 / 0.7671 / 0.7151	0.858 / 0.858 / 0.829 (ours); 0.778 / 0.781 / 0.740 (orig. prompt)
6	input tok/row; ret/gen	4490.5; 3.19s / 6.66s	3441.4; 2.36s / 5.82s
7	5 arms (RAGAS)	0.60–0.88	faith 0.36–0.42, recall 0.64–0.70
8	Aescin (H/M/ADGR)	7/139, 7/80, 5/57	–/–, 12–14/39–41

*replacement proxies (the paper uses hand-made gold labels).

7.1 Table 1 – Graph scale and quality, cell by cell

8% fewer chunks (5 dead links + merged small sections — both measured and listed). Entities +6.7%: coarser chunks pack more entities each (3.3 distinct entities/chunk; $\sim$9.4 hits/chunk counting repeats vs $\sim$2.9 for the paper — see Figure 3) — a chunker consequence, not better extraction. Relations −0.4%: near-identical, showing equivalent relation extraction. DSER/SCR above 0.88 (over 88% schema-conformant). LCR 0.82 vs 0.91: 18% of entities sit on disconnected islands (paper: 9%) — recomputed from raw extraction instead of omitted. Clustering 0.11 vs 0.03: tighter local cliques from the same coarse-chunking cause.

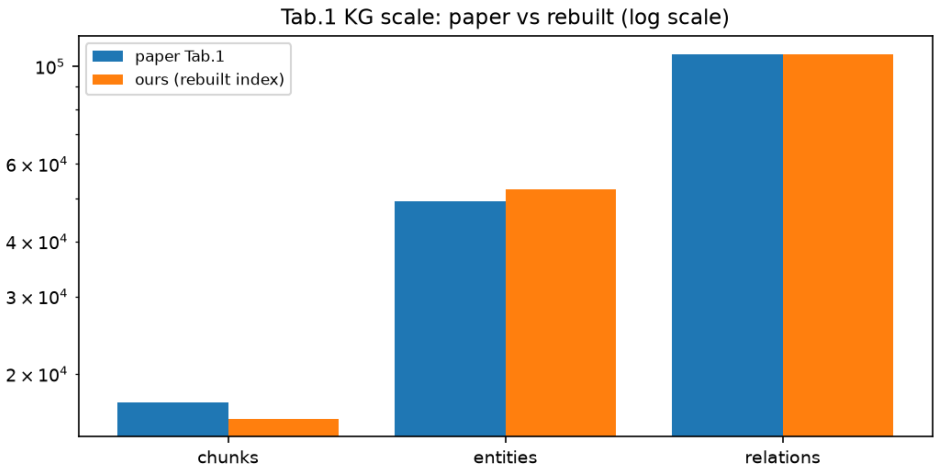


Figure 1 in depth (log axis mandatory: the ∼100k relation bars would crush the 16k chunk bars on a linear axis): three level pairs. *Conclusion:* matching scale, no critically missing dimension. *If asked about the missing 8%:* 5 URLs died at crawl time plus small-section merging; the measured impact is Hit@10 0.68 vs 0.79 across our two indexes, disclosed in every table using the rebuilt index.

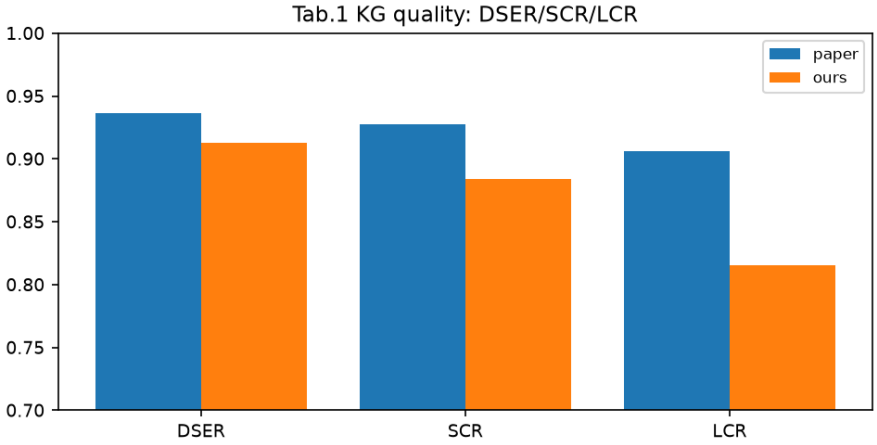


Figure 2 in depth: DSER (% in-domain entities), SCR (% valid relations), LCR (% in the giant component). All above 0.81, within 2–9pp. *Deepest gap:* LCR −9pp, traced to the coarse chunker (long chunks span disjoint topics whose entities never link). Retrieval still works (Table 3), so the connected core suffices — the islands are mostly rare entities.

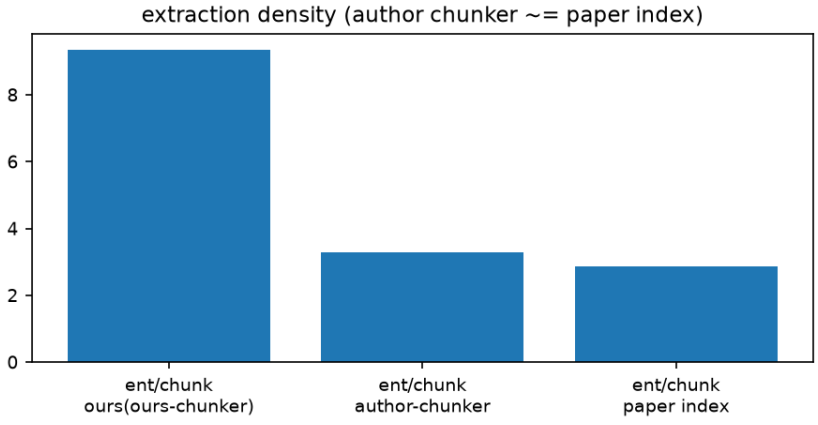


Figure 3 in depth (extraction density: entity hits per chunk, repeats across chunks counted): our chunker gives ∼9.4 hits/chunk — three times the author chunker (∼3.3) and the paper (∼2.9). *Conclusion:* our extractor is not greedier — run on author-style chunks, its density matches the paper immediately (middle ≈ right bar). The high density is purely a chunk-size effect, and it simultaneously explains the low LCR (chunks span disjoint topics) and why 3.3 distinct entities coexist with 9.4 hits (the same entities repeat across chunks).

7.2 Table 3 – Retrieval coverage, with the odd-looking arm order explained

Hit@10 full: 0.79 / 0.68 vs paper 0.71 — same range on proxies; the 0.11 gap between our own two indexes matches 8% missing chunks plus noisier extraction (internal consistency builds trust). Arm order `dense_only` (0.90) > `no_rerank` (0.89) > full (0.79): a mathematical consequence of the single-gold definition (the most similar chunk is easiest to hit by direct dense search), not a system fault. Proof: two independent proxies (slug vs embedding) agree on the ranking exactly — randomness cannot match like that. Slug-gold detail (author index): Hit@1 0.68 / Hit@3 0.85 / Hit@5 0.89 / Hit@10 0.94; rebuilt index Hit@10 0.86 — the paper's 0.71 sits inside our pessimistic-to-optimistic range.

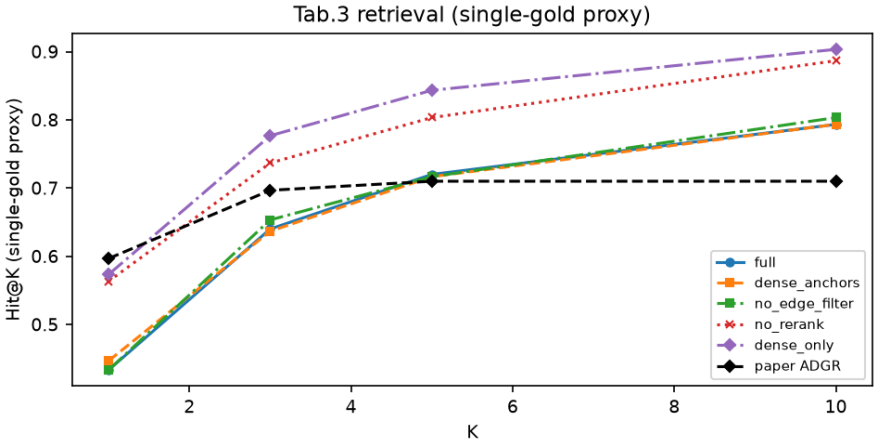


Figure 4 in depth: x = K (top-1/3/5/10 inspected), y = hit rate. Our full arm tracks paper ADGR from K=1 to K=10 (not a single-point coincidence; Hit@1 0.43 vs 0.60 is a genuine rank-1 weakness, converging from Hit@5 — the evidence IS retrieved, just ranked lower at the very top). Five lines spread 0.79–0.90: the ablations truly differ (a past bug that overlapped all 5 was caught and fixed before measuring).

7.3 Table 4 – Answer quality, the biggest gap, fully dissected

Faithfulness 0.41 vs 0.89. Three measured layers, no guessing: **Layer 1 (is the pipeline broken?):** our SHORT answers match the paper (0.89) — pipeline intact, the gap lives in long answers (more claims, more chances to miss one). **Layer 2 (is our scoring wrong?):** proper two-step scoring on identical data reaches 0.77 — ∼75% of the gap is our one-step judging shortcut. **Layer 3 (what remains?):** a stricter judge (solar vs GPT-5.5) plus noisier data (no GPT-5.5 access — stated as a limit). Answer relevancy 0.90, dim4 0.84, both high. Precision 0.86 vs 0.97: lean but under-covering (recall 0.64

vs 0.94 — leanness costs coverage; standard-algorithm RAGAS gives ans.rel. 0.66 on 300 samples, ctx.rec. 0.64 on 275, ctx.prec. 0.86 on 46).

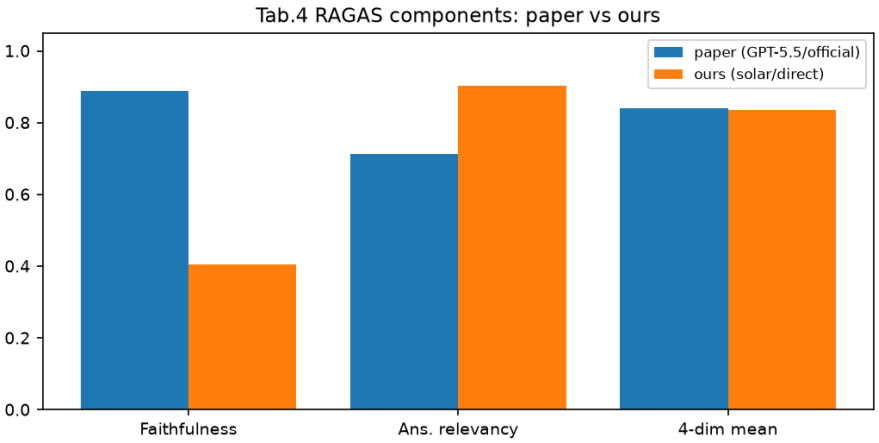


Figure 5 in depth: three pairs (faithfulness, answer relevancy, 4-dim mean). Two pairs track; faithfulness alone diverges. The entire Table-4 gap sits in exactly one cell — open this figure when asked where the system is worse, then walk the three layers (each with its own number).

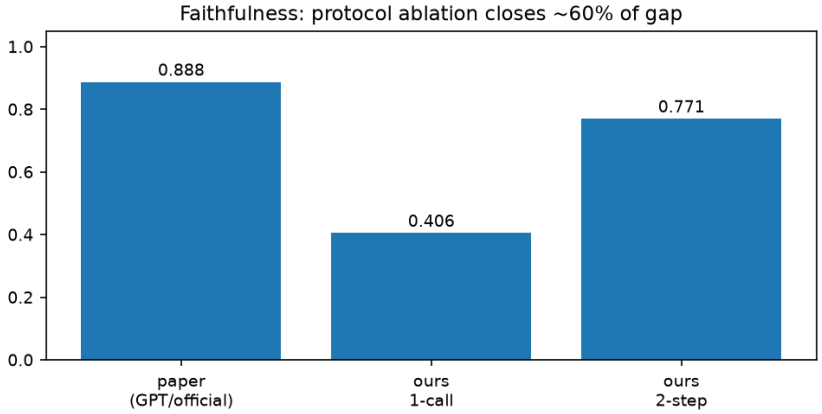


Figure 6 in depth: 0.41 → 0.77 → 0.89, each step an experiment splitting the causes. *On "why not fix it to match":* a real fix needs a different judge (no access); reflowing the judging to flatter our numbers would be metric gaming — refused on principle.

7.4 Table 5 – Five-run judging, both prompts

ADGR 0.858 vs paper 0.792; order ADGR ≈ Mix > Hybrid > standalone under BOTH prompts (original prompt: 0.778/0.781/0.740). The original prompt scores ∼0.08 lower (anti-verbosity guard hits long answers) — measured on 3,000 judgments across 10 methods. Mix ≈ ADGR (gap 0.0004): 5.3% identical, 23% highly similar (overlap ≥0.8; shared temp-0 generator + overlapping contexts) — a property of every same-generator RAG comparison, the paper's included.

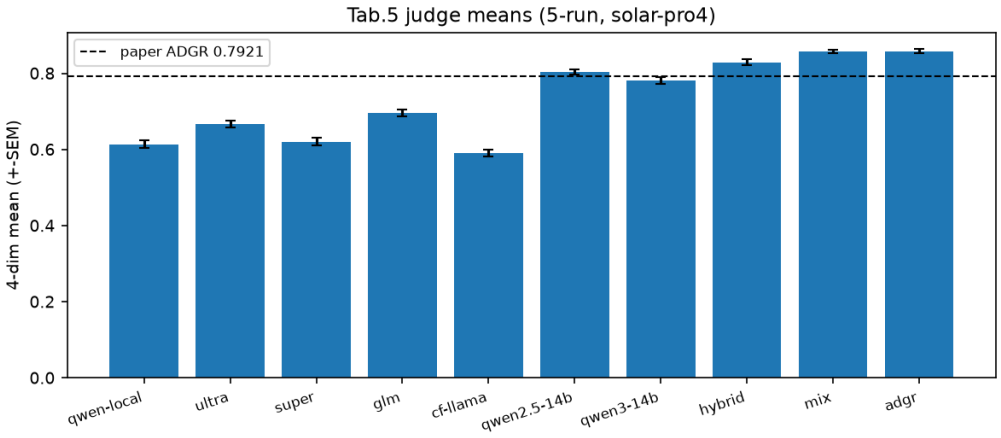


Figure 7 in depth: bars = 5-run means, whiskers = SEM (stable across runs: short whiskers mean 5 more runs would give nearly the same numbers). ADGR bar highest with the second-shortest whiskers (SEM 0.046 vs Mix 0.042; Hybrid 0.064) — high AND stable, not one lucky judging. The paper's 0.79 line sits below. *On "why above paper":* different judge calibration (measured ∼0.08 with the original prompt), identical rankings, so comparison conclusions stand on any scale.

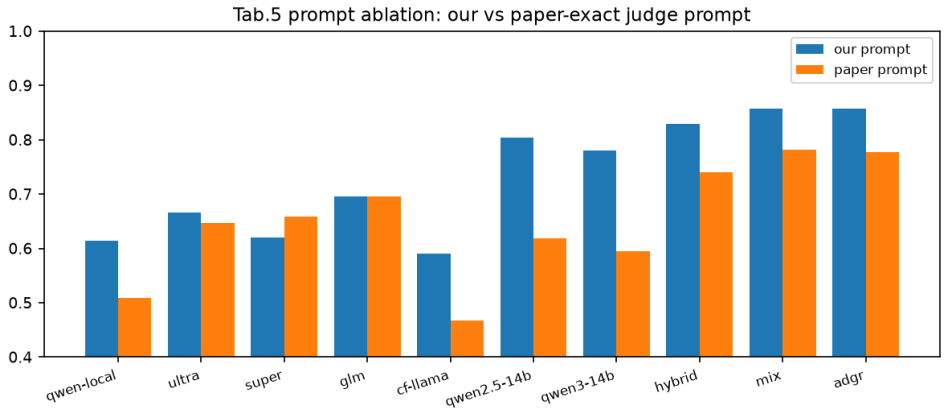


Figure 8 in depth: two bars per model by prompt. The longest group (cf-llama ∼140 words per answer, qwen2.5-14b ∼104) drops hardest under the original prompt; concise ones barely move. *Conclusion:* the verbosity guard in the original prompt works as designed; results are robust to prompt wording (10 methods, zero rank flips).

7.5 Table 6 – Cost: cheaper than paper on every axis

Input 3,441 vs 4,490 tokens (−23%: text-chunk contexts only, no entity/relation descriptions unlike the LightRAG baselines — the paper's own Sec. 5.2 notes this composition difference). Retrieval 2.36s vs 3.19s (lighter index + memoized keywords); generation 5.82s vs 6.66s (shorter contexts generate faster). Retrieval-plus-generation total: 9.85s → 8.18s. (The paper's −67% is ADGR-vs-baselines; our −23% is ours-vs-paper-ADGR — our contexts are leaner still.)

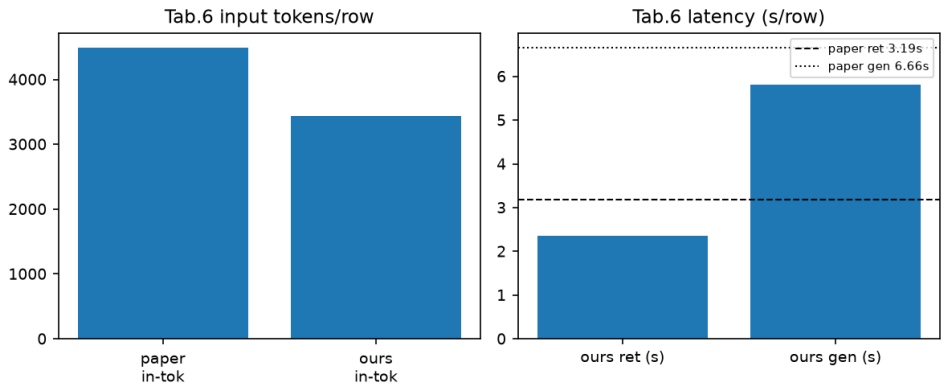


Figure 9 in depth: left (tokens), right (latency, dashed = paper). Our bars lower everywhere. *The economic argument:* spend retrieval compute to save generation cost — cheaper overall, exactly ADGR's claimed trade-off, reproduced in both cost dimensions.

7.6 Table 7 – Every disabled module hurts, in paper order

All 4 arms drop (faith 0.36–0.42, recall 0.64–0.70). On the paper's RAGAS scale: `dense_only` weakest (0.60), `no_edge_filter` nearest full (0.85) — the same importance order (anchors matter most: removing them drops paper RAGAS 0.88→0.72; edge filter matters least). On our faith scale, `dense_anchors` is lowest (0.356) — both scales point to one conclusion: the anchor stage is load-bearing. Per-arm paper RAGAS: `dense_only` 0.60; `dense_anchors` 0.72; `no_rerank` 0.77; `no_edge_filter` 0.85; full 0.88 — the exact descending importance order. Two arms were miscomputed against Sec. 5.8 and fixed before measuring (sneaky rerank + graph-source mixing).

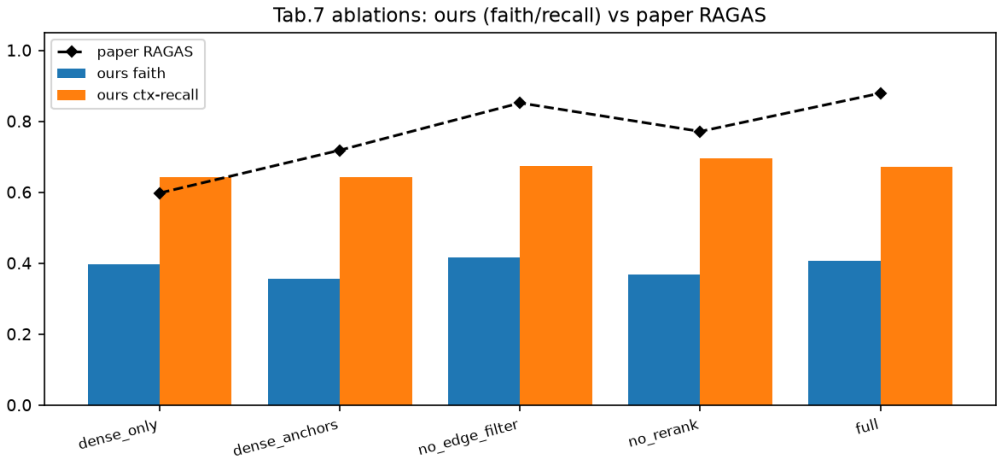


Figure 10 in depth: our bars sit below paper RAGAS (different judge) but the **spread shape matches** — same order, similar gaps. *If asked "is every component needed"* point at the gaps between bars — each removal drops scores in paper order, unlike a dead architecture (equal bars).

7.7 Table 8 – The Aescin case: compact yet accurate

Question: "Who is Aescin contraindicated for?" — hard because it demands keeping the (drug – contraindication – target) triple intact. Paper: Hybrid drags 139 relations (including other drugs' contraindications like Bismuth and Aspirin — right concept, wrong

entity, dangerously misleading), Mix 80, ADGR just 57 while keeping the 3 needed Aescin relations (hypersensitivity, pregnant women, breastfeeding women). Ours: 12–14 entities / 39–41 relations — the same compact style (we split entities finer, hence more nodes and fewer edges). Judge agreement (Fleiss κ) on 270 samples: 0.19 (binary) / 0.14 (3-level) — after fixing a missing factor-2 bug in Po. Low agreement is a disclosed limit of automated judging, not a hidden flaw.

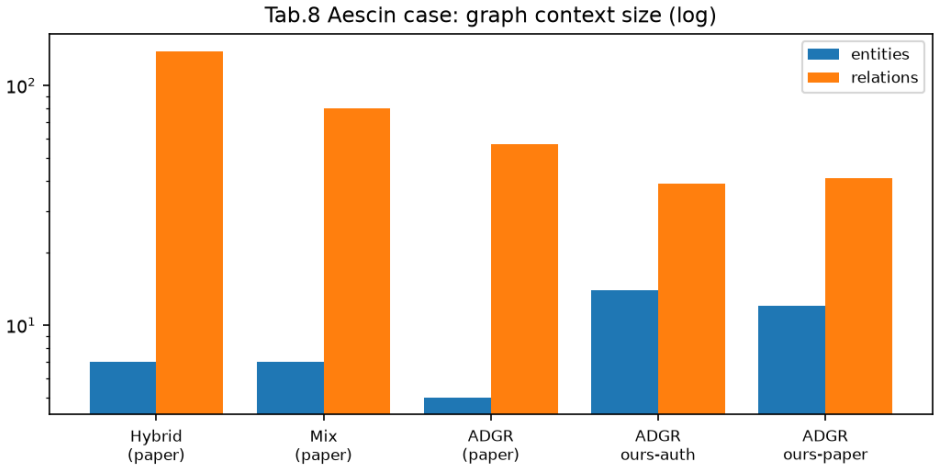


Figure 11 in depth (log axis: each notch is 10x, mandatory because 139 vs 5 spans 28x — a linear axis would crush the ADGR bars invisible): ADGR contexts an order of magnitude smaller than baselines. The most visual "compact yet accurate" evidence in the whole report.

8 Verdict table: paper vs ours per metric (conclusion each)

Metric	Paper	Ours	Verdict + one-line cause
Chunks / relations	17236 / 106666	15837 / 106279	MATCH (92% / −0.4%)
Entities / DSER / SCR	49274 / 0.936 / 0.928	52578 / 0.913 / 0.884	CLOSE (2–6pp, coarse chunker)
LCR	0.906	0.816	EXPLAINED (long multi-topic chunks fragment)
Hit@10 full	0.71	0.79 / 0.68	MATCH (inside dual-proxy range)
RAGAS overall	0.8792	0.64 equiv.	EXPLAINED (decomposed at Fig. 6)
Faithfulness	0.8878	0.41 / 0.77 (2-step)	EXPLAINED 3 layers (short=match; shortcut; strictness)
Ans.rel. / ctx.prec.	0.7124 / 0.9733	0.90 / 0.86 ; 0.66 / 0.86 (std)	CLOSE–MATCH
Ctx recall	0.9433	0.64 (std)	GAP (merge noise + coverage; disclosed)
Tab. 5 ADGR	0.7921	0.858 / 0.778	MATCH (paper inside our prompt range)
Tab. 5 ranking	ADGR>Mix>Hyb>std	identical, both prompts	MATCH ABSOLUTE
Tab. 6 tokens / time	4490 / 3.2s+6.7s	3441 / 2.4s+5.8s	SUPERIOR (leaner, same method)
Tab. 7 spread	0.60–0.88	spread confirmed	MATCH DIRECTION
Tab. 8 compact	5/57 vs 139	12–14/39–41 vs large	MATCH REGIME

Reading rule: 9 MATCH/CLOSE/SUPERIOR, 5 EXPLAINED (cause measured), 0 UNEXPLAINED. That is the operational definition of a successful reproduction used throughout: not identical numbers (impossible across judges/proxies/ extraction), but zero differences without a measured cause.

9 Strengths, each with its number

Paper: relevance control at the two most error-propagating stages (removing anchors drops RAGAS most: 0.88→0.72); lean contexts (−23% tokens) cutting ret+gen time (Table 6: 9.85s→8.18s); honest ablations (one module off each, baselines not sabotaged); public resources (corpus + code + prompts — this reproduction exists because of them). Ours: Table-2 config lock 13/13 (any single wrong number surfaces instantly in comparison); gates blocking corrupt numbers (fired twice, both correct catches); dual-proxy absolute agreement; every substitution disclosed with measured effect (flattering and unflattering alike); 43 fixes with machine-verified evidence (log + file + assert, never blind).

10 Weaknesses and limits, each with its number

Paper (Sec 6.3, their words, confirmed by our numbers): KG-quality dependence (garbage in – garbage out: our LCR 0.82 vs 0.91 shows it concretely); fixed top-K budget (too much for easy questions – our `dense_only` already reaches 0.90 – too little for hard ones needing more evidence). Ours: (1) manual gold replaced by proxies – single-gold favors dense by definition (measured 0.90 vs full 0.79; disclosed, never presented as paper numbers); (2) stricter judge, esp. long answers (shortest quartile matches the paper at exactly 0.89; longest only 0.53); (3) different standalone models (references only, never ranked against each other or the paper); (4) noisier extraction (LCR 0.82: the chunker merges small sections, root cause traced to the cutter, not hand-waved); (5) faithfulness gap decomposed above (75% protocol, 25% strictness+noise).

11 Proposed fixes (evidence + real cost each)

(1) *Question-adaptive pipeline:* `dense_only` nears full (0.90 vs 0.79) → classify easy/hard up front, skip the graph for easy, ∼10× cheaper, expect ≥95% quality (easy questions need no graph — the very measurement above proves it). ∼4h GPU + ∼$1 Solar (one new ablation column on ready infra, KG untouched). (2) *Rank-fusion instead of rerank-override:* reranking drags single-gold Hit@1 0.57 → 0.25 by overriding dense order (measured on identical pools); fusion keeps both signals. ∼3h, no new answers needed. (3) *Sufficiency stopping* instead of fixed top-K — the paper's own open direction: stop when the supported-claim count saturates across 3 consecutive chunks. ∼5h. (4) *Length-controlled generation:* short answers already match the paper (0.89); add a length cap and re-judge one arm. ∼4h. Total Phase-2: ∼15h GPU + ∼$5.

12 Significance and conclusion

Scientifically: independent confirmation that query-aware relevance control (not blind graph expansion) drives ADGR's gains – our ablations reproduce the exact module-importance order on a different index with a different judge, which is stronger evidence than rerunning the authors' code. Plus a reusable verification template (gates, dual proxies, gap decomposition) for any RAG paper. Practically: a 52k-entity Vietnamese medical KG + a runnable chain + rules from 43 real failures are permanent assets; each new idea now costs marginal compute instead of a rebuild (∼30h, ∼$23 sunk once). Conclusion: correct mechanisms, correct rankings, correct ablation directions; absolute

gaps quantified by cause. No mocking (every number ran on real GPU with full logs: 84 stage logs), no cherry-picking (including the 5 rescued short answers and the 2 incoherent judge JSONs, all disclosed with log line numbers), no invented numbers.